

实验五：中间代码生成与优化

一、实验概述

本实验是编译原理实验系列的最后一个实验。在实验四中，你的编译器已经能够对程序进行完整的词法、语法和语义检查。本实验的目标是：在经过语义检查的 AST 基础上，生成中间表示（IR），并实现若干代码优化，从而完成一个端到端的编译器前端到中端的完整流程。

中间代码是源语言与目标机器之间的桥梁，它独立于具体机器，便于进行优化与移植。本实验将带你完成从 AST 到 IR 的翻译，并体会优化对代码质量的提升。

二、实验目标

- 理解中间表示（三地址码 / 栈式字节码 / AST 解释）的作用与设计
- 掌握语法制导翻译，将 AST 翻译为中间代码
- 理解控制流（条件、循环）的翻译方法（标签与跳转）
- 实现常见的代码优化，并能评估优化效果

三、实验内容

任务一：中间代码生成（必做）

在前序实验得到的 AST 基础上，生成中间代码。后端策略由你自选并在报告中说明：三地址码（TAC）、栈式字节码、或基于 AST 的直接解释执行均可。

1. 命令行接口

```
./compiler codegen input.src
```

输出中间代码（文本或 JSON 格式均可）。以三地址码为例，对 $a = b + c * 2$ 可生成：

```

t1 = c * 2
t2 = b + t1
a  = t2

```

控制流的翻译示例（if (x > 0) y = 1;）：

```

    t1 = x > 0
    if_false t1 goto L1
    y = 1
L1:

```

2. 必做功能

(1) 表达式翻译

- 将算术、关系、逻辑表达式翻译为中间代码，正确分配临时变量
- 保持运算符优先级与结合性（继承自实验三的 AST）

(2) 控制流翻译

- if / if-else 翻译为条件跳转与标签
- while / for 等循环翻译为跳转结构

(3) 函数与调用翻译（如你的语言有函数）

- 函数定义、参数传递、返回值、函数调用的翻译

(4) IR 格式文档化

你的 IR 指令集需在报告中文档化（指令类型、操作数形式、标签与跳转约定等）。

任务二：代码优化（必做）

在中间代码上实现至少 2 - 3 种优化，并通过命令行选项启用，展示优化前后的对比。

```
./compiler codegen input.src --optimize
```

可选的优化（实现其中至少 2 - 3 种）：

优化	说明	示例
常量折叠	编译期计算常量表达式	$t = 2 * 3 \rightarrow t = 6$

常量传播	用已知常量替换变量引用	$x=5; y=x+1 \rightarrow y=6$
死代码消除	删除不会被使用的计算/不可达代码	删除从未使用的临时变量赋值
公共子表达式消除	复用已计算的相同子表达式	$a*b$ 计算一次复用
代数化简	利用代数恒等式化简	$x*1 \rightarrow x, x+0 \rightarrow x, x*0 \rightarrow 0$

需在报告中给出优化前后的中间代码对比，并说明优化带来的指令数/计算量变化。

任务三：中间代码执行（选做，加分项）

实现一个 IR 解释器/虚拟机，执行你的中间代码并输出程序运行结果，验证翻译的正确性。

```
./compiler run input.src
```

任务四：必考场景验证（必做）

使用正向必考场景程序（S1 - S5，场景定义见《必考场景集（扩充版）》）验证中间代码生成：

- 各场景程序必须成功生成中间代码
- 若实现了任务三的执行器，展示运行结果的正确性

在演示与报告中展示场景程序的中间代码（优化前后）。

四、交付物清单

序号	交付物	文件/格式	说明
1	中间代码生成器源代码	源代码文件	建议与前序实验共用代码库
2	构建/运行说明	README.md	如何编译和运行
3	IR 指令集设计文档	ir-design.md 或写入报告	指令类型、操作数、跳转约定
4	优化前后对比	tests/ 或写入报告	覆盖各项已实现优

			化
5	场景中间代码	scenarios/S1.ir.txt ~ S5.ir.txt	正向场景，含优化前后
6	AI 协作对话记录	conversation/ 目录 (Markdown 或导出文件)	完整对话 + 每轮工作纪要
7	实验报告	report.pdf / report.docx	含完整功能演示流程 (图文)

五、验收方式

本实验采用过程化验收，重点考察真实的开发过程以及功能实现的一致性，而非仅看最终代码。验收由助教完成，包含以下四个环节。四个环节相互印证：对话记录、实验报告、源代码三者必须能够对应起来。

环节一：实验演示检查

- 学生现场演示（或提交完整录屏），展示编译器本阶段功能的完整运行流程，覆盖本实验全部必做功能。
- 演示需同时体现：正常输入的正确处理，以及错误输入的检测与报告。
- 助教可在演示过程中临时更换输入用例、要求你解释某段代码的作用或某个设计决策的理由。

环节二：AI 协作过程记录检查

- 提交开发全过程中与智能体（Agent IDE / AI 助手，如 Claude Code、Cursor、CodeBuddy 等）的完整对话记录。
- 记录需包含每一轮交互的三部分：（1）你向智能体提出的问题或指令；（2）智能体对该轮对话的完整响应；（3）该轮的工作纪要——你做了什么决策、为什么这样做、如何验证结果。
- 工作纪要不要求包含代码输出本身，重点是设计思路、决策理由与验证过程。

- 课程鼓励使用 AI 工具，但你必须能够独立解释每一行代码与每一个设计选择。完整、连贯的对话记录是你真正参与开发的证据。

环节三：实验报告检查

- 实验报告必须包含完整的功能演示流程（图文并茂）：对每一项功能，给出运行截图加文字说明，完整展示从输入到输出的过程。
- 报告还需涵盖：设计决策、关键算法实现思路、遇到的问题及其解决方法。

环节四：代码审核

- 助教审核源代码，要求与环节二的对话记录、环节三的报告演示流程保持一致：
 - 代码中实现的功能，应能在报告的演示流程中找到对应的展示；
 - 代码的实现方式与思路，应能由对话记录与工作纪要解释其来源。
- 若出现明显不一致（例如代码中存在报告未演示、对话记录也无法解释的功能），将影响本实验成绩。

六、实验报告要求

实验报告是验收的核心材料之一，必须包含以下内容：

（1）完整功能演示流程（图文并茂）

- 逐项展示本实验的每个功能：给出输入示例、运行命令、运行截图与输出结果，并配文字说明。
- 演示流程需与源代码、AI 对话记录一致（详见第五部分验收方式）。

（2）设计与实现说明

- 你选择了哪种后端策略（三地址码 / 字节码 / 解释）？为什么？
- 你的 IR 指令集是如何设计的？给出指令定义。
- 表达式与控制流是如何翻译的？举例说明（含临时变量与标签分配）。
- 你实现了哪些优化？给出优化前后的中间代码对比与效果分析。
- 整个编译器（实验一到实验五）的完整流程回顾，以及各阶段的接口设计。

（3）AI 协作过程小结

- 简述你如何与智能体协作完成本实验：哪些部分由你主导设计、哪些借助 AI 实现、你如何验证 AI 产出的正确性。

七、参考资料

- Aho, Lam, Sethi, Ullman. Compilers: Principles, Techniques, and Tools (Dragon Book), Chapter 6, 8, 9
- Appel. Modern Compiler Implementation, Chapter 7, 8, 10, 17
- Muchnick. Advanced Compiler Design and Implementation (优化进阶参考)
- LLVM IR、Java 字节码等真实中间表示的文档（仅供参考）